

think

A systems course for people coming home. Offline, from the ground up.

Penn Porterfield · 2026 · working title · for my Next Chapter mentors

think is a small, self-contained course that teaches the low level of a computer, from true zero, on a machine that is not allowed to touch the internet. It is one offline Rust program. It teaches how a computer actually works, at your own pace, and it ends with DOOM booting on a screen the learner drove from the metal up. Below is the problem it answers, why I am the one to build it, and how it connects to reentry.

43%

lower odds of returning to prison for people in correctional education¹

\$4–\$5

saved for every \$1 spent on prison education¹

27%

unemployment for the formerly incarcerated, ~5× the general rate³

83%

of organizations weigh open-source contributions when hiring¹⁰

THE PROBLEM

Every coding program for justice-impacted people teaches web development. The Last Mile teaches JavaScript and React. Persevere teaches full-stack. Justice Through Code teaches Python. These are good programs doing real good, and I respect them. But web development is the most crowded corner of the industry, the place everyone who did a bootcamp already stands, so it does little to set anyone apart. It is also the shrinking, lower-paid lane: web developers earn a median of \$90,930 against \$133,080 for software developers, a gap of about \$42,000,⁶ and entry-level software employment for people in their early twenties has fallen roughly 20% since 2022.⁷ And teaching web dev inside a facility is a bit of a simulacrum. You cannot use the live internet,

so the program mirrors a fake one in for you to practice against. You come out one step removed from the real thing.

Meanwhile I could not find one prison or reentry program that teaches systems, embedded, or kernel work. Not one. That is a real white space, and there is a plain reason it stayed empty: low-level work usually needs hardware, and hardware does not clear facility review. So the most durable, least crowded part of the field is the one part nobody is teaching to the people who could most use a way to stand out.

WHY I SHOULD BUILD THIS

Because I have lived exactly this, and I am already doing the hard part.

I taught myself on a graphing calculator in the honor block of a maximum security prison, then worked through the “You Don’t Know JavaScript” series on a locked-down library browser because it was the only machine I could reach. I know what it is to want this and have to assemble it out of scraps. I spent time as the assistant for the math GED, too, and I know what it is to watch someone finally get it.

I am not proposing this from the sidelines. Right now I am building the capstone it teaches toward, myself: an out-of-tree Rust driver for an SPI display with DOOM running on it, on a BeaglePlay, traced on a Saleae logic analyzer. So I know the material is real and current. Rust became a first-class part of the Linux kernel in version 7.0 this year, and there is still no in-tree Rust SPI abstraction as of the 7.2 development kernel,⁹ so this sits on a live frontier, not a history lesson. Rust is also the most admired language among developers, at 72%.⁸

And it is buildable. The whole low-level stack is simulated in software, so it clears the wall the incumbents could not: one static binary, no network, no installer, and a check in the build that keeps the language engineering and not hacking. It is cheap, it is self-contained, and the first phase is already built and passing tests. I can open the classroom and show it running today.

HOW IT CONNECTS TO REENTRY

Reentry is the stretch right before you get out and the first months after, when you have the least footing and the most to prove. The stakes are not abstract. About 453,200 people were released from prison in 2023,⁵ and within five years 76.6% of released people are rearrested.² A big part of why is work: the formerly incarcerated face about 27% unemployment, nearly five times the general rate,³ and a criminal record alone cuts an applicant’s callbacks roughly in half.⁴

Education is one of the few things with hard evidence that it changes this. People who take part in correctional education have 43% lower odds of returning to prison, vocational training raises the odds of employment by 28%, and RAND estimates every dollar spent saves four to five in reincarceration costs.¹ The whole course is built around the reentry window: you learn the fundamentals inside, on a machine with no internet, and the week you get home you turn that into something real.

The point of reentry is walking out with something you can show, and this is where thunk is different from a web-dev course. It ends in tangible artifacts a person can put on a resume:

- a device driver they wrote, and DOOM running on a panel because of it;
- and, once they are out, their first real contribution to open source: a documentation fix, a bug in a tool they use, and for the ones who go far, a Linux kernel patch, which is the path I am walking myself.

That last one matters more for people coming home than for almost anyone else, and the numbers agree that it is real currency: 83% of organizations weigh open-source contributions when they hire,¹⁰ and a developer's public contribution activity measurably rises when they are on the job market, because it works as a signal.¹⁰ A merged contribution is public and permanent. It is your name on real work that shipped, in a place that does not ask about your record. When a background check might quietly close a door, a history of open-source contributions is a door that is already open, and nobody can take it back.

So I want to kindle a small community around this: a handful of people doing small, well-scoped kernel fixes together, plugged into the on-ramps that already exist for this (kernelnewbies, the Linux Kernel Mentorship Program, Outreachy) instead of reinventing them. What makes it fit our situation is the culture of the work itself.

Linux development is a meritocracy. The review judges the patch, not the person. Nobody on the mailing list asks about your record, and a correct patch gets merged with your name on it. The culture is exacting and often blunt, but it is blind to your past and ruthless only about your code. For people carrying a record, that is not a slogan. It is the mechanics.

So the arc matches the moment. Inside, you build the skill. At release, you build the record, together with a few other people who get it. Both are things you can point to when someone asks what you can do.

WHAT I AM ASKING FOR

A yes, and the room to build this as my justice-impacted work. Your feedback where I am wrong; I ship better code when someone tells me it is not up to snuff, and I have learned to take that well. And an introduction where you have one, to Next Chapter leadership, to The Last Mile, to Saleae.

SOURCES

Verified July 2026. Program-reported outcomes (e.g. The Last Mile, Persevere) are omitted here in favor of independent sources; a randomized evaluation of The Last Mile is underway but unpublished.

1. RAND Corporation, *Evaluating the Effectiveness of Correctional Education: A Meta-Analysis* (Davis et al., RR-266), 2013. 43% lower odds of recidivism; 28% higher employment odds for vocational training; \$1 → \$4–\$5 saved (break-even model). rand.org/pubs/research_reports/RR266.html
2. U.S. Bureau of Justice Statistics, *Recidivism of Prisoners Released in 30 States in 2005* (NCJ 244205), 2014. 67.8% rearrested within 3 years, 76.6% within 5. bjs.ojp.gov
3. Prison Policy Initiative, *Out of Prison & Out of Work*, 2018. 27% unemployment (only national estimate; 2008 data). prisonpolicy.org/reports/outofwork.html
4. Devah Pager, “The Mark of a Criminal Record,” *American Journal of Sociology* 108(5), 2003. A record cut callbacks from 34% to 17%. Replicated since.
5. U.S. Bureau of Justice Statistics, *Prisoners in 2023 — Statistical Tables* (NCJ 310197), 2025. 453,200 released from state and federal prison. bjs.ojp.gov
6. U.S. Bureau of Labor Statistics, *Occupational Employment & Wage Statistics*, May 2024. Software developers median \$133,080; web developers \$90,930. bls.gov/ooh
7. Stanford Digital Economy Lab, “Canaries in the Coal Mine?” (ADP payroll data), 2025. Employment for software developers ages 22–25 down ~20% from its 2022 peak.
8. Stack Overflow Developer Survey 2025. Rust the most-admired language, 72.4%. survey.stackoverflow.co/2025
9. LWN.net (Dec 2025) and kernel.org: Rust became a core part of the kernel in Linux 7.0 (2026); the official kernel Rust crate at v7.2-rc2 has no *spi* module. lwn.net/Articles/1049831 · rust.docs.kernel.org
10. 2024 Open Source Software Funding Survey (GitHub, the Linux Foundation, and Harvard). 83% of organizations say open-source contribution is at least somewhat important for prospective hires; peer-reviewed labor research finds contribution activity rises during job search as a signal.